CSC413/CSE315

Design of Operating System

Programming Assignment – Spring 2026

Due on: 26th April 2026

Please read the following information very carefully.

- The assignment is divided into 2 sections: a **mandatory** section and an **optional** section. The mandatory section is worth 100 marks. *The optional section is not mandatory!!!!*
- Ensure you submit on time. Late submissions may be penalized.
- **Answer** every question to the **best of your abilities** and **do not resort to plagiarism** as <u>you will be asked to present your answers at **viva**</u>. Failure to show competence according to your answers will be marked as plagiarism and dealt with accordingly.
- The optional section contains difficult and tough questions which most may not be able to answer properly. **Only attempt these questions after you are done with the mandatory set.** That being said, successfully answering any optional question will merit you much praise and significant consideration in the grading for this course.
- Submissions are individual. *However, you may participate in a group to solve the optional section problems. Clearly state all students who worked in the group to merit this bonus.*
- Attempt to **answer every question in** the **mandatory** section in its own code file/folder.
- Ensure your deliverables are in a **zip** file with the following prefix.
  `“<Section Number>_<ID>_<Course Name>_<Student Name>.zip”`
  ex: 3_2120117_CSC413-Ezhar Uddin.zip
  ex: 1_2091012_CSE315_Rezwan_Ahmed.zip
  This helps with automation. Your submission may not be graded if this prefix does not match.
- **WARNING: No code submission will be accepted inside a PDF or Word document**. Ensure all your code exists as code/project files that can be easily executed/run on demand.
- If needed, any textual justifications or descriptions of how you solved a problem can be accepted, either through code comments or if necessary, a separate PDF or Word document. But the questions will be asked primarily in viva.
- For **viva**, ensure you have a **laptop** that is at the very least, capable of running all the applications in the mandatory section. You may need the following:
  - An IDE capable of compiling Java 8/JavaFX [ex: IntelliJ/NetBeans]
  - Linux Desktop Environment + Build tools [ex: Ubuntu/Red Hat]
  - Windows Desktop Environment + An IDE capable of compiling WIN32 applications [ex: CodeBlocks]

  If you do not have this, you do not need to appear for the viva.
- All students who worked in a group to solve an optional problem must be present at the viva together.
- You have to appear at the viva only for the questions which you have answered successfully. If you did not submit the programming assignment you do not need to appear for the viva.

Mandatory Section

YOU SHOULD ANSWER EVERY QUESTION IN THIS SECTION. SUBMIT THE CODE/PROJECTS VIA ZIP SUBMISSION AND PREPARE TALKING POINTS FOR THE QUESTIONS IN THE BULLETED LISTS.

A. Word Counter

The parent process forks multiple children. Each child counts words in a specific string (or file) and sends the count back to the parent via a pipe. The parent then sums up the results.

Project Enhancements

- **Dynamic Tasks**: Allow the user to input multiple file names and have the shell fork a child for each file.
- **Error Reporting**: Have the child send an error code (like -1) through the pipe if a file cannot be found.

B. The Elusive Cursor [20]

Instead of breaking the clipboard, you write a background application that makes the mouse cursor "jump" or move slightly away whenever the user tries to click a specific area (like the Start button) or simply jitters the mouse every few seconds.

Key Win32 API functions you would use:

- **SetCursorPos(x, y)**: To forcefully move the mouse.
- **GetCursorPos(&point)**: To track where the user is currently aiming.
- **SetTimer()**: To create the "annoying" interval where the movement happens.
- **GetSystemMetrics()**: To ensure the mouse stays within screen boundaries (so it doesn't disappear).
- **RegisterHotKey()**: To provide yourself a "secret" way to kill the program (essential for "harmless" pranks).

You should also be able to describe**:**

1. How you installed and used the Windows SDK
2. How you solved the implementation
3. Documentation/resources needed

C. Multi-threaded Image Grayscale Converter or a **Parallel Matrix Adder**. **[20]**

The parent process creates multiple threads. Each thread is assigned a specific set of rows in two large matrices. It adds those rows together and stores the result in a third matrix.

a. **Data Partitioning**: Thread A only touches rows 0–9; Thread B only touches rows 10–19.
b. **Independent Writes**: No two threads ever write to the same memory address.
c. **Read-Only Inputs**: The source matrices are never modified, so multiple threads can read them simultaneously without issues.

Project Enhancements

- **Performance Testing**: Use the time command in Linux to compare how much faster the 4-thread version is vs. a 1-thread version.
- **Dynamic Threading**: Let the user input the number of threads (e.g., ./matrix_add 8) and calculate the row chunks dynamically.
- **Error Check**: Ensure the number of rows is divisible by the number of threads, or handle the "remainder" rows in the last thread.

D. Implement a **Time-Sliced Multilevel Queue (MLQ) Simulator**. **[20]**

In this version, the CPU doesn't just wait for Queue 1 to be empty before moving to Queue 2. Instead, it allocates a specific **percentage of total CPU time** to each queue.

Example Configuration:

- **Queue 1 (System):** 50% CPU time, uses **Round Robin (RR)**.
- **Queue 2 (Interactive):** 30% CPU time, uses **Shortest Job First (SJF)**.
- **Queue 3 (Batch):** 20% CPU time, uses **First-Come, First-Served (FCFS)**.

Step 1: Define the Process and Queue Structures

You need a way to store process data and group them into distinct queues based on their priority range.

struct Process {

```
  int pid;
  int arrival_time;
  int burst_time;
  int remaining_time;
  int priority; // e.g., 0-10=Q1, 11-20=Q2, 21-30=Q3
};
// Use separate vectors or linked lists for each queue
vector<Process> Q1, Q2, Q3;
```

Step 2: Implement the Inter-Queue Time Slicer

Instead of strict preemption, use a global "Time Slice" (e.g., 10 seconds total) and divide it among the queues. [10]

1. **Allocate Time**: Assign 5 seconds to Q1, 3 seconds to Q2, and 2 seconds to Q3.
2. **Execute Q1**: Run processes in Q1 using its specific algorithm (RR) for exactly 5 seconds.
3. **Context Switch**: Move the CPU to Q2 and run its processes (SJF) for 3 seconds.
4. **Execute Q3**: Run processes in Q3 (FCFS) for 2 seconds.
5. **Repeat**: Loop back to Q1.

Step 3: Handle Individual Queue Logic

Each queue must manage its own processes independently during its allocated time window.

- **Round Robin (Q1)**: Use a small time quantum (e.g., 2ms) to cycle through system tasks.
- **Shortest Job First (Q2)**: Sort processes by burst_time so the shortest tasks finish first.
- **FCFS (Q3)**: Simply execute processes in the order they arrived.

Key Performance Metrics to Calculate

Your simulator should output a table or Gantt chart showing:

- **Waiting Time**: (Completion Time - Arrival Time - Burst Time).
- **Turnaround Time**: (Completion Time - Arrival Time).
- **Response Time**: Time from arrival to the first time the process gets the CPU.

E. Real-Time Task Scheduler & Schedulability Analyzer. [20]

Implements both static and dynamic priority algorithms. Unlike standard simulators, a real-time version focuses on **deadlines** and **periodicity**, which are critical for systems like flight controllers or medical devices.

Implement a simulator that allows users to toggle between two major real-time scheduling theories:

- **Rate Monotonic (RM)**: A static priority algorithm where tasks with shorter periods get higher priority.
- **Earliest Deadline First (EDF)**: A dynamic priority algorithm where the task closest to its absolute deadline is executed first.

1. Implementation Steps

Step 1: Define Periodic Tasks

In real-time systems, tasks are often periodic. You must define a structure to track these properties:

- **Period (P):** The fixed interval at which the task recurs.
- **Execution Time (E):** The CPU time needed per period.
- **Deadline (D):** Usually equal to the period (D = P).
- **Arrival Time:** The start of the current period.

Step 2: Implement Rate Monotonic (RM) Logic

Assign priorities once at the beginning:

i. Sort tasks by their period (P).
ii. Assign the highest priority to the task with the smallest P.
iii. In the simulation loop, always run the highest-priority task that is "ready" (its arrival time has passed and it has remaining execution time).

Step 3: Implement Earliest Deadline First (EDF) Logic

Update priorities dynamically at every clock tick:

i. At each scheduling event, calculate the **absolute deadline** for all ready tasks (Absolute Deadline = Arrival Time + Relative Deadline).
ii. The scheduler selects the task with the smallest absolute deadline.
iii. If a new task arrives with a deadline earlier than the currently running one, the current task is **preempted**.

Step 4: Add Schedulability Testing

Before running the simulation, your program should check if the task set is even possible to schedule.

- **RM Bound:** Total utilization $U = \sum \frac{E_i}{p_i}$ must be $\leq n(2^n - 1)$. For many tasks, this is approximately 0.69.
- **EDF Bound:** Total utilization U can be up to 1.0 (100%).

2. Core Simulation Loop (Logic)

```
// Pseudocode for the real-time scheduler loop
int current_time = 0;
while (current_time < hyper_period) { // hyper_period = LCM of all task periods
  // 1. Activate tasks whose periods have started
  for (auto& task : task_list) {
    if (current_time % task.period == 0) {
      task.ready = true;
      task.remaining_time = task.execution_time;
      task.absolute_deadline = current_time + task.period;
    }
  }
  // 2. Select task based on mode (RM vs EDF)
  Task* best_task = nullptr;
  if (mode == "EDF") {
    best_task = select_earliest_deadline(task_list);
  } else { // RM
    best_task = select_highest_static_priority(task_list);
  }

  // 3. Execute and check for deadline misses
  if (best_task) {
    best_task->remaining_time--;
    if (current_time + 1 > best_task->absolute_deadline) {
      cout << "Deadline Missed for Process " << best_task->id;
    }
  }
  current_time++;
}
```

Algorithms to Compare

- **Rate Monotonic (RM)**: Assign static priorities based on periods (shorter period = higher priority).
- **Earliest Deadline First (EDF)**: Assign dynamic priorities based on the absolute deadline of the current task instance.

Example Test Case for Your Project

Use the following task set to verify your simulator's accuracy:

Task	**Execution Time (C_i)**	**Period (T_i)**	**Utilization (U_i)**
τ_1	2	5	0.40
τ_2	4	10	0.40
τ_3	1	20	0.05
Total			**0.85 (85%)**

Expected Analysis Results:

I. **RM Schedulability**: For n=3, the bound is $3(2^{\frac{1}{3}} - 1) \approx 0.779$. Since $0.85 > 0.779, the RM test is inconclusive; your simulator must determine if it fails or succeeds.

II. **EDF Schedulability**: Since 0.85 <1, EDF is guaranteed to be feasible.

Optional Section

ONLY ATTEMPT THIS SECTION IF YOU HAVE COMPLETED THE MANDATORY SECTION. ANSWERING ANY ONE PROBLEM IS SUFFICIENT BUT FEEL FREE TO SOLVE MORE IF YOU ARE CAPABLE. YOU CAN USE ANY AND EVERY RESOURCE AVAILABLE TO YOU TO SOLVE THESE PROBLEMS, SHORT OF PLAGIARISM.

A. Introduction to Linux Kernel Programming: In this project, you will learn how to create a kernel module and load it into the Linux kernel.

This project will require the use of a Linux OS and possibly the source code of the kernel you are using. Although you may use any text editor to write these C programs, you will have to use the terminal application to compile the programs, and you will have to enter commands on the command line to manage the modules in the kernel.

As you'll discover, the advantage of developing kernel modules is that it is a relatively easy method of interacting with the kernel, thus allowing you to write programs that directly invoke kernel functions. It is important for you to keep in mind that you are indeed writing kernel code that directly interacts with the kernel. That normally means that any errors in the code could crash the system!

I. Kernel Modules Overview

The first part of this project involves following a series of steps for creating and inserting a module into the Linux kernel. You can list all kernel modules that are currently loaded by entering the command

```
$ lsmod
```

This command will list the current kernel modules in three columns: name, size, and where the module is being used.

```c
/* simple.c: a file that demonstrates Linux modules */
#include <linux/init.h>
#include <linux/kernel.h>
#include <linux/module.h>

/* This function is called when the module is loaded. */
int simple_init(void)
{
    printk(KERN_INFO "Loading Kernel Module\n"); return 0;
}

/* This function is called when the module is removed. */
void simple_exit(void)
{
    printk(KERN_INFO "Removing Kernel Module\n");
}

/* Macros for registering module entry and exit points. */

module_init(simple_init);
module_exit(simple_exit);
MODULE_LICENSE("GPL");

MODULE_DESCRIPTION("Simple Module");
MODULE_AUTHOR("SGG");
```

The program above is a very basic kernel module that prints appropriate messages when it is loaded and unloaded. The function `simple_init()` is the module entry point, which represents the function

that is invoked when the module is loaded into the kernel. Similarly, the `simple_exit()` function is the module exit point— the function that is called when the module is removed from the kernel.

The module entry point function must return an integer value, with 0 representing success and any other value representing failure. The module exit point function returns void. Neither the module entry point nor the module exit point are passed any parameters. The two following macros are used for registering the module entry and exit points with the kernel:

```
module init(simple_init);
module exit(simple_exit);
```

Notice in the figure how the module entry and exit point functions make calls to the `printk()` function. `printk()` is the kernel equivalent of `printf()`, but its output is sent to a kernel log buffer whose contents can be read by the `dmesg` command. One difference between `printf()` and `printk()` is that `printk()` allows us to specify a priority flag, whose values are given in the `<linux/printk.h>` include file. In this instance, the priority is `KERN_INFO`, which is defined as an informational message.

The final lines — `MODULE_LICENSE()`, `MODULE_DESCRIPTION()`, and `MODULE_AUTHOR()`— represent details regarding the software license, description of the module, and author. For our purposes, we do not require this information, but we include it because it is standard practice in developing kernel modules and compilation will fail without a valid license.

This kernel module *simple.c* is compiled using the Makefile accompanying the source code with this project. To compile the module, enter the following on the command line:

```
$ make
```

The compilation produces several files. The file *simple.ko* represents the compiled kernel module. The following step illustrates inserting this module into the Linux kernel.

II. Loading and Removing Kernel Modules

Kernel modules are loaded using the `insmod` command, which is run as follows:

```
$ sudo insmod simple.ko
```

To check whether the module has loaded, enter the `lsmod` command and search for the module *simple*. Recall that the module entry point is invoked when the module is inserted into the kernel. To check the contents of this message in the kernel log buffer, enter the command

```
$ sudo dmesg
```

You should see the message "Loading Module." Removing the kernel module involves invoking the `rmmod` command (notice that the '.ko' suffix is unnecessary):

```
$ sudo rmmod simple
```

Be sure to check with the `dmesg` command to ensure the module has been removed. Because the kernel log buffer can fill up quickly, it often makes sense to clear the buffer periodically. This can be accomplished as follows:

```
$ sudo dmesg -c
```

Proceed through the steps described above to create the kernel module and to load and unload the module. Be sure to check the contents of the kernel log buffer using `dmesg` to ensure that you have followed the steps properly.

As kernel modules are running within the kernel, it is possible to obtain values and call functions that are available only in the kernel and not to regular user applications. For example, the Linux include file `<linux/hash.h>` defines several hashing functions for use within the kernel. This file also defines the constant value `GOLDEN_RATIO_PRIME` (which is defined as an `unsigned long`).

This value can be printed out as follows:

```
printk(KERN_INFO "%lu\n", GOLDEN_RATIO_PRIME);
```

As another example, the include file `<linux/gcd.h>` defines the following function.

```
unsigned long gcd(unsigned long a, unsigned b);
```

which returns the greatest common divisor of the parameters a and b.

III. Project Deliverables

Once you can correctly load and unload a module, write a kernel module to complete the following additional steps:

- Print out the value of `GOLDEN_RATIO_PRIME` in the `module_init()` function.
- Print out the greatest common divisor of 3,700 and 24 in the `module_exit()` function.

- **Source Code**: The .c file and Makefile.
- **Report**: A document explaining the OS concepts covered (e.g., kernel space vs. user space, dynamic kernel extensions), your implementation details, challenges faced, and testing procedures.
- **Demonstration**: Showing the module loading and unloading successfully in a Linux environment

B. **CSE 213/CSC305 Project Extension:** In your previous study of Object-Oriented Programming, you were asked to develop a real-world Java GUI application that sought to solve some IT/digital needs of some business or organization. Modify that project so it is capable of RMI and client-server communications.

Your task is to investigate and point out aspects in your project that could be serviced by some remote server instead of being implemented on the client.

Having a method implemented on the client makes it vulnerable to being exposed by reverse engineering or hacking: hence sensitive functions or functions that work with sensitive data are often implemented through remote APIs. Alternatively, a method may also be extremely computationally intensive, to the point where it is unfeasible to have it be processed client-side: such methods are also qualified to be RMI, and for such applications, the server is equipped with a lot of processing power to handle such calls on a large volume.

I. Submission

Once you have documented such use cases, implement working RMI or RPC systems to service some key methods. A significant amount of work must be done for this optional question to be considered completed. You are also required to explain

- What functions/actions have been implemented via remote execution methods
- Justifications for implementing said functions as RMI
- Demonstrate execution of RMI

II. Why this is necessary

Through revisiting past projects, students can identify areas for improvement, address any shortcomings, and implement new techniques or technologies that they may have learned since the project's completion. Moreover, extending previous projects provides an opportunity for students to apply their knowledge in a practical context, reinforcing key concepts and fostering a deeper appreciation for the subject matter.

Working incrementally on your projects also allows you to showcase your growth and development over time to future interviewers in job interviews and gives you more prominence in your career as a developer. As a result, we feel it is beneficial if you complete this optional and strongly recommend you take it seriously.

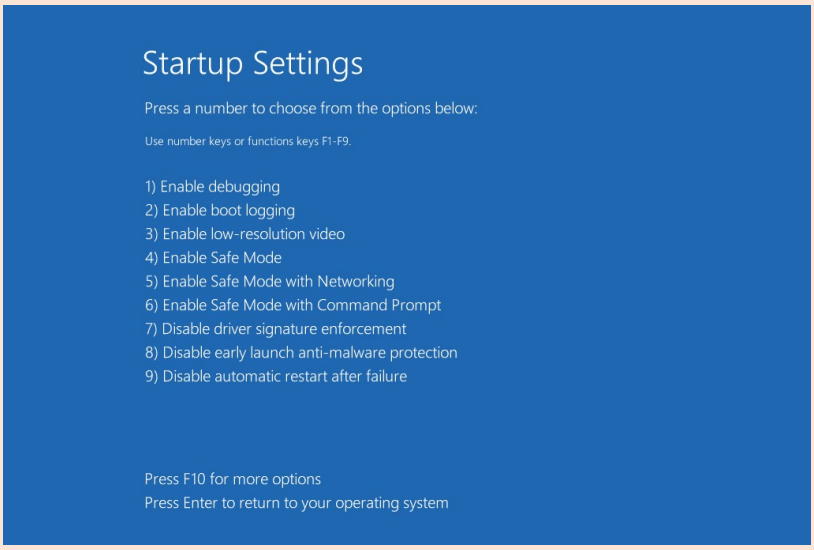
C. **WIN32 Driver Programming**: Demonstrate your ability to handle desktop applications programming and comprehend API and their documentation by writing a simple Windows driver.

Keep in mind WIN32 drivers are very similar to how Linux kernel modules work. No support will be provided for programming and compiling the driver: you must discover and find resources on the Internet that will help you to program a driver on your own. Once you have managed to do that,

I. Running the Driver:

The driver you have compiled should be something like `driver.sys`. However, getting any arbitrary driver to run on Windows is quite difficult compared to on Linux. This is because Windows employs **driver signing** to ensure that unsafe or dangerous code in drivers cannot be freely executed. The kernel-mode code signing policy enforces that kernel-mode drivers must be signed for the driver to load. Getting your drivers signed is only possible if you are a large enough company which can enter into an agreement with Microsoft or go through a strenuous approval process and have your drivers be thoroughly tested by WHQL. As amateur and niche OS programmers we want to avoid this hassle.

To allow your unsigned driver to execute, you must first restart your PC into **Startup Settings/Advanced Boot Options** by holding Shift while clicking on Power > Restart, or pressing F8 when your computer is booting. Then press F7 or 7 to **disable driver signature enforcement**.

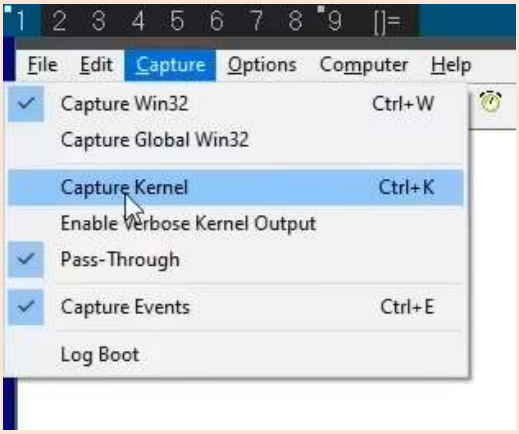


This allows unsigned drivers to temporarily execute during that session. After this, we need to create and start a service that uses our driver (this is somewhat like an entry-point for our driver, code in our driver needs to first start executing through some service). Open the command prompt with administrative privileges and type in

```
> sc
DESCRIPTION:
        SC is a command line program used for communicating with the
        Service Control Manager and services.
...
> sc create myDriver binPath= C:\Path\to\Driver\driver.sys type= kernel
[SC] CreateService SUCCESS
> sc start myDriver
The driver will execute without any prompt. Check DebugView for output.
> sc stop myDriver
This will unload the driver. Check DebugView for output (if you have handled it).
> sc delete myDriver
[SC] DeleteService SUCCESS
```

II. View output

Windows is an operating system primarily built for its customers, the masses. To provide an easy, seamless, and user-friendly experience for most people, there are very few built-in tools that allow programmers to tinker around and manipulate the OS. There is no built-in analogue of the Linux Kernel log viewer `dmesg`. Hence to view the output of the driver we will need an additional software called DebugView from Sysinternals. Run DebugView as an administrator before you start the driver in the section above.



Then click on **Capture > Capture Kernel** and **Enable Verbose Kernel Output** to ensure you can see the output produced by your driver.

III. Submission

Once you can correctly load and unload a driver, write a driver and complete the following additional steps:

- Print out "Hello Kernel! <your name>" using `DbgPrint()` in the driver entry function that appears only in the kernel log output captured by DebugView.
- Print out "Goodbye Kernel!" when your driver is unloaded that appears only in the kernel log output captured by DebugView.
- [**EXTREMELY DIFFICULT**] Prove your driver is executing code in kernel mode by calling the kernel mode functions `ZwCreateFile()` and `ZwWriteFile()` and make some random file: show the `NTSTATUS` and `IO_STATUS_BLOCK` struct's contents to show this file was written successfully. [You are considered successful even if your system crashes]

You are expected to be able to demonstrate the compilation, explain problems/challenges you faced, and show successful loading/unloading of the driver on your system.